

# Fullstack Development

# Part 2: The battle of the meta frameworks

| Next.js vs Astro vs TanStack Start

# What meta-frameworks do

- **File-Based Routing**
  - Files and folders define the URL structure of your app.
- **Server-Side Rendering (SSR)**
  - Builds HTML on a server to make pages load fast and work well with search engines.
- **Static Site Generation (SSG)**
  - Builds HTML at build time, so the server doesn't have to do any work when a user visits.

# What meta-frameworks do

- **Data Fetching**
  - Provides a way to fetch data from a database or API and pass it to your components.
- **Server Actions / RPC**
  - Lets you call server-side code from the client without writing your own API endpoints.
- **Optimization:** Speeds up images, scripts, and code automatically

# Elevator pitches

- `Astro` : A content-first web framework that ships zero JavaScript by default.
- `Next.js` : A production framework for React, architected server-first around Server Components.
- `TanStack Start` : A production framework for React, architected client-first around a full SPA router — with SSR built in.

# Modern rendering taxonomy

- **Static Site Generation (SSG) / Static Prerendering / Partial Prerendering (PPR)**
  - Build-time HTML, no server needed.
- **Server-Side Rendering (SSR)**
  - HTML is built on the server per request.
  - Optional streaming
- **SSR + Hydration / Isomorphic Rendering**
  - HTML is built on the server, then the client takes over and makes it interactive.
- **Client-Side Rendering (CSR)**
  - HTML is built in the browser, no server rendering.

# Rendering modes - comparison

| Name       | When it runs | Where           | Delivery                          |
|------------|--------------|-----------------|-----------------------------------|
| SSG        | Build time   | Server          | One complete HTML file            |
| SSR        | Request time | Server          | One response (or streamed)        |
| Isomorphic | Request time | Server + Client | One response, then rehydrated     |
| CSR        | Request time | Client          | Blank shell, then client-rendered |



# Takeaway

| Name       | Astro           | Next.js          | TanStack Start |
|------------|-----------------|------------------|----------------|
| SSG        | Astro Component | Server Component | *              |
| SSR        | Server Island   | Server Component | *              |
| Isomorphic | Client Island   | Client Component | Any components |
| CSR        | *               | *                | *              |

\* = not the "main" behavior (in my opinion), but possible as an option.

# Takeaway notes

- **Astro default** is SSG, everything else is opt-in.
- **Next.js default** is SSR, everything else is opt-in.
  - In V15 SSG/SSR is decided by compiler (with exceptions). In V16, with partial prerendering, SSG/SSR is decided by directives ( `'use cache'` ).
- **TanStack Start default** is isomorphic (SSR + SPA hydration), everything else is opt-in.

# Todo app

# The todo app

- Astro
  - `git clone https://github.com/fullstack-69/ls-astro.git`
- Next.js
  - `git clone https://github.com/fullstack-69/ls-nextjs.git`
- TanStack Start
  - `git clone https://github.com/fullstack-69/ls-tanstack-start.git`
- Same UI (Pico CSS), same database, same features.
- Only the **architecture** differs.

# Three parts, three timestamps

| Component | What it does                                       | Question it answers        |
|-----------|----------------------------------------------------|----------------------------|
| Nav       | prints a timestamp                                 | When was the shell made?   |
| Greeting  | reads <code>user-agent</code> + prints a timestamp | Was this per-request?      |
| Todo      | list + delete (interactive)                        | Where does the data enter? |

If a timestamp never changes on refresh → it was **baked at build time**.

# Shared data layer

`src/db/index.ts` — identical in all three projects

```
export async function getTodos() {
  await ensureTodosTable();
  await sleep(DB_LATENCY); // artificial latency, so we can SEE the loading states
  const result = await client.execute(
    "SELECT id, todoText FROM todos ORDER BY rowid ASC",
  );
  return result.rows.map((row) => toTodo(row));
}

export async function deleteTodo(id: number) { ... }
```

- libSQL / SQLite ( `file:app.db` ).
- **Server-only code.** It must never reach the browser bundle.

# Diagram

<https://link.excalidraw.com/I/9PltHIQHZMD/4zVYOIJ54Ee>

# DX comparison

| From a perspective of a developer who knows React and SPA frameworks.



# Astro — two familiar things, kept separate

- The Astro component is closer to a classic template language (Handlebars, EJS, Pug) than to React.
  - `---` fence = data fetching. Template below = HTML output. No hooks, no lifecycle.
  - Fits my intuition immediately because it is the old-school model.
- The React island is **100% ordinary React**
  - It is a fully separate render tree with its own bundle.
  - No new React concepts. `useState`, `useEffect`, callbacks — all work as expected.

# Next.js — familiar syntax, unfamiliar rules

- Next repurposes **ordinary-looking React syntax** to mean something environment-dependent.
  - A string literal ( `'use client'` / `'use server'` / `'use cache'` ) is intercepted by the compiler and silently changes what the file is allowed to do and where it runs.
- It looks identical to the React I know. It behaves differently.
  - You cannot `import` a Server Component inside a `'use client'` file.
  - No `useState` in a server Component.

# TanStack Start — explicit over implicit

- No magic string. No compiler rewriting the meaning of a component file.
- The server/client boundary is drawn by **which function you're inside**, not by a hidden annotation on the file.
- Writing React component code is the **same everywhere**. The only difference is whether you call a server function or not.
- `TanStack Router` and `Query` are libraries SPA developers already have muscle memory for — `Start` reuses them rather than replacing them.

# References

This part is AI-generated from my codebase. Use it for reference, not as a tutorial.

# Astro

# Astro component

```
ls-astro/src/components/Greeting.astro
```

```
---
import { UAParser } from "ua-parser-js";

const uaHeader = Astro.request.headers.get("user-agent") || "";
const result = new UAParser(uaHeader).getResult();
const timeStr = new Date().toLocaleTimeString();
---

<article class="card">
  <h1>Hello, {result.browser.name}!</h1>
  <p>Generated at: {timeStr}</p>
</article>
```

# Astro component (anatomy)

- **Component script** — between the `---` fences.
  - Plain TS/JS, runs **only on the server**, never shipped to the browser.
  - Can `import` server-only modules freely (db, `node:fs`, secrets).
- **Component template** — below the fences.
  - HTML + `{expressions}`.
  - Renders **once**, to a string. No re-render, no hooks, no state.
- Output: pure HTML. **Zero JS.**

# Astro page

```
ls-astro/src/pages/index.astro
```

```
---
import Layout from "@components/Layout.astro";
import Nav from "@components/Nav.astro";
import Greeting from "@components/Greeting.astro";
import TodoReact from "@components/Todo.tsx";
---

<Layout>
  <Nav />
  <Greeting server:defer>
    <article slot="fallback"><p>Loading <span aria-busy="true"></span></p></article>
  </Greeting>
  <TodoReact client:load />
</Layout>
```



# Astro page (routing)

- **File-based routing** — anything in `src/pages/` becomes a route.
  - `src/pages/index.astro` → `/`
  - `src/pages/blog/[slug].astro` → `/blog/:slug`
- A page is just an `.astro` component that returns a full document.
- `src/components/Layout.astro` is the html shell; children arrive via `<slot />`.

```
<body class="container">  
  <slot />  
</body>
```

# Island architecture

The page is a **sea of static HTML**. Interactivity lives in isolated **islands**.

static HTML • 0 KB JS

<Nav/>

static

Greeting

← server island

<TodoReact client:load />

← client island (React)

*Each island hydrates independently. No island → no JS.*

# Two kinds of islands

```
<Nav />                                <!-- 1. plain: static HTML, 0 KB JS -->
<Greeting server:defer />              <!-- 2. server island: HTML fetched at request time -->
<TodoReact client:load />              <!-- 3. client island: React, hydrated in browser -->
```

|              | Renders where       | JS to browser     |
|--------------|---------------------|-------------------|
| <Nav />      | server, at build    | none              |
| server:defer | server, per request | none              |
| client:load  | server + browser    | React + component |

# Client islands — hydration directives

| Directive                   | Hydrates when                    |
|-----------------------------|----------------------------------|
| <code>client:load</code>    | immediately on page load         |
| <code>client:idle</code>    | browser is idle                  |
| <code>client:visible</code> | island scrolls into viewport     |
| <code>client:media</code>   | CSS media query matches          |
| <code>client:only</code>    | never SSR'd — client render only |

Islands are the **only** JS in the bundle: `dist/client/_astro/ToDo.*.js`.

# Server islands

```
ls-astro/src/pages/index.astro
```

```
<Greeting server:defer>
  <article slot="fallback">
    <h1>Greeting</h1>
    <p>
      Loading <span aria-busy="true"></span>
    </p>
  </article>
</Greeting>
```

- Page HTML ships **immediately** with the `fallback`.
- The island is then fetched from `/_server-islands/Greeting` and swapped in.
- The static page can sit on a CDN; only the island hits the origin.

# Astro rendering mode

```
ls-astro/astro.config.mjs
```

```
export default defineConfig({  
  adapter: node({ mode: "standalone" }),  
  integrations: [react()],  
});
```

- `output: "static"` **is the default** — pages are prerendered to HTML at build.
- The `adapter` adds a server, for islands / actions / SSR pages.

# Opting out, per page

```
ls-astro/src/pages/some-page.astro
```

```
---  
export const prerender = false; // this page now renders per request  
---
```

| Mode                           | When                                                |
|--------------------------------|-----------------------------------------------------|
| prerendered (default)          | content is the same for everyone                    |
| <code>prerender = false</code> | page depends on the request (auth, personalization) |

Astro makes you opt **in** to the server. Next.js and Start make you opt **out**.

# Proof: what actually got built

```
dist/
├── client/
│   ├── index.html          ← the whole page, prerendered
│   └── _astro/TODO.*.js    ← the only JS shipped
└── server/
    └── chunks/Greeting.mjs ← the server island, rendered on demand
```

- Nav's timestamp is **frozen** in `index.html` (build time).
- Greeting's timestamp changes on **every request**.
- Todo's data arrives **after** hydration.



# Astro Actions

```
ls-astro/src/actions/index.ts
```

```
import { defineAction } from "astro:actions";
import { z } from "astro/zod";
import { getTodos, deleteTodo } from "../db/index.ts";

export const server = {
  getTodos: defineAction({
    handler: async () => await getTodos(),
  }),
  deleteTodo: defineAction({
    input: z.object({ id: z.number() }), // runtime validation
    handler: async ({ id }) => {
      await deleteTodo(id);
      return null;
    },
  }),
};
```

# Calling an Action from an island

```
ls-astro/src/components/Todo.tsx
```

```
import { actions } from "astro:actions";

const { data, error } = await actions.getTodos();
if (error) {
  console.error(error);
  return;
}
setTodos(data);

// delete
await actions.deleteTodo({ id: todo.id });
```

# Calling an Action from an island

- Type-safe RPC — no `fetch`, no URL, no manual JSON.
- Returns `{ data, error }`; errors are **values**, not exceptions.
- The `import` looks local, but it compiles to a POST to `/_actions/...`.

# Astro summary

- Static HTML is the **default**; everything else is opt-in.
- `client:*` → client island (React ships).
- `server:defer` → server island (no JS ships).
- `actions` → the typed client → server boundary.
- Mental model: **an HTML document that occasionally contains an app.**

# Next.js

# App Router

```
ls-nextjs/src/app/  
├── layout.tsx    → root layout (wraps every route)  
├── page.tsx      → the route "/"  
└── favicon.ico
```

- **Folders define routes; files define UI.**
- Reserved filenames: `page`, `layout`, `loading`, `error`, `not-found`, `route`, `template`.
- `app/blog/[slug]/page.tsx` → `/blog/:slug`

# Root layout



```
ls-nextjs/src/app/layout.tsx
```

```
export const metadata: Metadata = {
  title: "Next.js",
  description: "A Next.js app",
};

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en">
      <body className="container">{children}</body>
    </html>
  );
}
```

- Owns `<html>` / `<body>` . Persists across navigation (does not re-render).

# The page

```
ls-nextjs/src/app/page.tsx
```

```
export default function Home() {  
  return (  
    <>  
      <Nav />  
      <Suspense  
        fallback={  
          <article>  
            <h1>Greeting</h1>  
            <p>Loading...</p>  
          </article>  
        }  
      >  
        <Greeting />  
      </Suspense>  
      <Todo />  
    </>  
  );  
}
```

# The page

- Compare with `index.astro` — same three components, same shape.
- `<Suspense>` replaces Astro's `server:defer` + `slot="fallback"`.

# Server Component

```
ls-nextjs/src/components/Greeting.tsx
```

```
import { headers } from "next/headers";

const Greeting: FC = async () => {
  const headersList = await headers(); // server-only API
  const uaHeader = headersList.get("user-agent") || "";
  const browser = new UAParser(uaHeader).getResult().browser.name;
  const timeStr = new Date().toLocaleTimeString();

  return (
    <article className="card">
      <h1>Hello, {browser}!</h1>
      <p>Generated at: {timeStr}</p>
    </article>
  );
};
```

# Server Component (rules)

- **The default.** No directive needed — every component in `app/` is a Server Component.
- Can be `async` and `await` directly in the component body.
- Can touch the database, secrets, `headers()`, `cookies()`.
- Ships **zero JS** — the browser receives the rendered output, not the code.
- Cannot use `useState`, `useEffect`, `onClick`, or any browser API.

| This is the direct analogue of the `---` fence in an `.astro` file.

# Client Component

```
ls-nextjs/src/components/Todo.tsx
```

```
"use client";
import { useEffect, useState, useTransition } from "react";
import { actionDeleteTodo, actionGetTodos } from "@actions";

const TodoComponent: FC = () => {
  const [todos, setTodos] = useState<Todo[]>([]);
  const [loading, startTransition] = useTransition();

  useEffect(() => { startTransition(fetchTodos); }, []);
  ...
};
```

# Client Component (rules)

- `"use client"` at the top of the file marks the **boundary**, not the file alone.
  - Everything imported *below* it also becomes client code.
- Still **server-rendered once** (for the HTML), then **hydrated** in the browser.
- Gets hooks, event handlers, browser APIs.
- Astro's `client:load` = Next's `"use client"` — but Next has **no** `client:visible`; hydration is not scheduled per-component.



# Server Actions

```
ls-nextjs/src/actions/index.ts
```

```
"use server"; // Marks ALL exports in this file as Server Actions

import { deleteTodo, getTodos } from "@/db";

export async function actionGetTodos() {
  return await getTodos();
}

export async function actionDeleteTodo(id: number) {
  await deleteTodo(id);
  return null;
}
```

# Calling a Server Action

```
ls-nextjs/src/components/ToDo.tsx
```

```
const data = await actionGetTodos(); // looks like a local call...
setTodos(data);

await actionDeleteTodo(todo.id); // ...is actually a POST to the server
```

- The bundler replaces the function body with an **RPC stub + an ID**.
- Throws on error → wrap in `try/catch` (Astro returns `{ data, error }` instead).
- ⚠ No built-in input validation: `id: number` is a *compile-time* claim only. Astro's `input: z.object(...)` and TanStack's `.validator()` do this for you.

# Partial Prerendering (PPR)

One route = a **static shell** delivered instantly + **dynamic holes** streamed in.

```
ls-nextjs/next.config.ts
```

```
const nextConfig: NextConfig = {  
  reactCompiler: true,  
  cacheComponents: true, // Enables 'use cache' behavior  
};
```

- Previously `experimental.ppr`; in Next 16 it is driven by `cacheComponents`.

# PPR — the static half

```
ls-nextjs/src/components/Nav.tsx
```

```
const Nav: FC = async () => {  
  "use cache"; // → prerendered into the shell  
  const timeStr = new Date().toLocaleTimeString();  
  return (  
    <nav>  
      ...<a href="#NA">Generated at: {timeStr}</a>...  
    </nav>  
  );  
};
```

- "use cache" = "this output is not per-request, bake it."
- Its timestamp is **frozen at build**, exactly like Astro's `Nav.astro`.

# PPR — the dynamic half

```
ls-nextjs/src/app/page.tsx
```

```
<Suspense fallback={<p>Loading...</p>}>  
  <Greeting /> { /* calls headers() → cannot be prerendered */ }  
</Suspense>
```

- `headers()` makes `Greeting` **dynamic**, so it must not block the shell.
- Next prerenders the page **with the fallback in place**, then streams the real HTML into the hole.
- Forget the `<Suspense>` and the build fails — the boundary is the contract.

# PPR vs. Astro server islands

|              | Astro                                      | Next.js                                                         |
|--------------|--------------------------------------------|-----------------------------------------------------------------|
| Static shell | <code>.astro</code> page, prerendered      | route shell, prerendered                                        |
| Dynamic hole | <code>&lt;Greeting server:defer&gt;</code> | <code>&lt;Suspense&gt;&lt;Greeting/&gt;&lt;/Suspense&gt;</code> |
| Placeholder  | <code>slot="fallback"</code>               | <code>fallback={...}</code>                                     |
| Delivery     | second HTTP request                        | streamed in the same response                                   |
| Marked by    | directive on the tag                       | <i>inferred</i> from dynamic API usage                          |

# Next.js summary

- Server is the default; the client is opt-in via `"use client"`.
- One language, one component model, one mental model — React everywhere.
- Static / dynamic is **inferred** from what your code touches, not declared.
- Mental model: **an app that happens to render on the server first.**

# TanStack Start



# File-based routing

```
ls-tanstack-start/src/routes/  
├── __root.tsx      → the shell (html/head/body)  
├── index.tsx       → /  
├── ssr_prerendered.tsx → /ssr_prerendered  
└── ssr_awaited.tsx  → /ssr_awaited
```

- Files → routes, but **compiled into a typed route tree**: `src/routeTree.gen.ts`.
- Generated automatically by the Vite plugin. **Never edit it.**

# The root route

```
ls-tanstack-start/src/routes/__root.tsx
```

```
export const Route = createRootRoute({
  head: () => ({
    meta: [{ title: "TanStack Start" }, { charset: "utf-8" }],
    links: [{ rel: "stylesheet", href: picoStyles }],
  }),
  shellComponent: RootDocument,
});

function RootDocument({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <head>
        <HeadContent />
      </head>
      <body>
        {children}
        <Scripts />
      </body>
    </html>
  );
}
```

# A route

```
ls-tanstack-start/src/routes/index.tsx
```

```
export const Route = createFileRoute("/")({
  component: Home,
  loader: async () => {
    const nav = await getNavTime();
    const greeting = await getGreeting();
    return { nav, greeting };
  },
});
```

- A route is a **declarative object**: a path, a component, and its data.
- Not a default export — the router imports `Route` by convention.

# The route's component

```
ls-tanstack-start/src/routes/index.tsx
```

```
function Home() {  
  const state = Route.useLoaderData(); // fully typed, no generics needed  
  
  return (  
    <main className="container">  
      <Nav timeStr={state.nav.timeStr} />  
      <Greeting  
        browser={state.greeting.browser}  
        timeStr={state.greeting.timeStr}  
      />  
      <Todo />  
    </main>  
  );  
}
```

# Loaders

- A route **declares its own data** in `loader`.
  - Runs on the server during SSR, and on the client during SPA navigation.
- `Route.useLoaderData()` is typed from the loader's return — no manual types.
- Contrast:
  - Next: each Server Component fetches its own data.
  - Astro: the page frontmatter fetches.
  - Start: **the route** fetches, then passes down as props.

# Typed links

```
ls-tanstack-start/src/components/Nav.tsx
```

```
import { Link, useLocation } from "@tanstack/react-router";

<Link
  to="/ssr_prerendered"
  className={isActive("/ssr_prerendered") ? "active" : ""}
>
  SSR Prerendered
</Link>;
```

- `to` is checked against the generated route tree — a typo is a **type error**.
- Client-side navigation (SPA), with `defaultPreload: "intent"` prefetching on hover.

# Isomorphic components

```
ls-tanstack-start/src/components/Greeting.tsx
```

```
interface GreetingProps {  
  browser: string;  
  timeStr: string;  
}  
  
const Greeting: FC<GreetingProps> = ({ browser, timeStr }) => (  
  <article className="card">  
    <h1>Hello, {browser}!</h1>  
    <p>Generated at: {timeStr}</p>  
  </article>  
);
```

**No** "use client". **No** "use server". **No** client:load.



# Isomorphic — what it means

- There is **one** kind of component. It runs on the server (SSR) *and* in the browser (hydration).
- Every component ships to the client. There is no "server-only component".
- So the server/client boundary is **not** the component — it is the **server function**.

| Framework | Boundary drawn at                                        |
|-----------|----------------------------------------------------------|
| Astro     | the island tag ( <code>client:load</code> )              |
| Next.js   | the file ( <code>"use client"</code> )                   |
| Start     | the <b>function call</b> ( <code>createServerFn</code> ) |

# Server functions

```
ls-tanstack-start/src/actions/index.ts
```

```
import { createServerFn } from "@tanstack/react-start";
import { getRequestHeaders } from "@tanstack/react-start/server";

export const getGreeting = createServerFn({ method: "GET" }).handler(
  async () => {
    const header = getRequestHeaders();
    const ua = header.get("user-agent") ?? "";
    return {
      browser: new UAParser(ua).getResult().browser.name || "Unknown",
      timeStr: new Date().toLocaleTimeString(),
    };
  },
);
```

- **Explicit method** — GET is cacheable, POST is a mutation.

# Server functions — validation

```
ls-tanstack-start/src/actions/index.ts
```

```
const deleteTodoSchema = z.object({ id: z.number() });

export const actionDeleteTodo = createServerFn({ method: "POST" })
  .validator(deleteTodoSchema) // runtime-validated input
  .handler(async ({ data }) => {
    await deleteTodo(data.id);
    return null;
  });
```

- Builder chain: `createServerFn` → `.middleware()` → `.validator()` → `.handler()`.
- The `.handler` body is **stripped from the client bundle**; the call becomes an RPC.

# Calling a server function

ls-tanstack-start/src/components/ToDo.tsx

```
import { useServerFn } from "@tanstack/react-start";

const getTodos = useServerFn(actionGetTodos);
const deleteTodo = useServerFn(actionDeleteTodo);

const data = await getTodos();
await deleteTodo({ data: { id: todo.id } }); // note the { data: ... } envelope
```

- Direct call works too; `useServerFn` additionally integrates with the router (redirects, error handling, invalidation).

# Prerendering

ls-tanstack-start/vite.config.ts

```
tanstackStart({
  prerender: {
    enabled: true,
    crawlLinks: true, // follow links and prerender those too
    filter: ({ path }) =>
      ["/ssr_prerendered", "/ssr_awaited_prerendered"].includes(path),
  },
});
```

- **Off by default** — the opposite of Astro.
- At build, Start renders the chosen routes to static HTML.

# Prerendering — the catch

ls-tanstack-start/src/actions/index.ts

```
import { staticFunctionMiddleware } from "@tanstack/start-static-server-functions";

export const getNavTime = createServerFn({ method: "GET" })
  .middleware([staticFunctionMiddleware]) // result baked at build time
  .handler(async () => ({ timeStr: new Date().toLocaleTimeString() }));
```

- Prerendering the *page* is not enough — the **server function results** must be frozen too.
- Otherwise the static HTML re-fetches on hydration and the timestamp jumps.

# Streaming a slow part

```
ls-tanstack-start/src/routes/ssr_awaited.tsx
```

```
loader: (async () => ({
  nav: await getNavTime(),
  greetingPromise: getGreeting(), // NOT awaited → sent as a promise
})),
(
  <Suspense fallback={<span aria-busy="true"></span>}>
    <Await promise={state.greetingPromise}>
      {(greeting) => (
        <Greeting browser={greeting.browser} timeStr={greeting.timeStr} />
      )}
    </Await>
  </Suspense>
));
```

- Start's answer to `server:defer` / PPR: **don't await it in the loader.**

# TanStack Start summary

- The **router** is the framework; SSR is a feature bolted on top of a real SPA router.
- One component model; the boundary is `createServerFn`.
- End-to-end type safety: routes, links, loader data, server-fn input & output.
- Nothing is static unless you ask.
- Mental model: **a SPA that can also render on the server.**



# A better framing than MPA vs SPA

What does the framework **assume by default**, and what has to be **specially marked** to deviate?

- **Astro** — assumes **nothing runs**. Static HTML at build. Everything else opts in.
- **Next.js** — assumes the **server runs it fresh, per request**. Static is earned as an optimisation.
- **TanStack Start** — assumes the **server runs it, then hands the whole tree to the client**.

# The "exception" axis

|                   | Astro                            | Next.js                             | TanStack Start               |
|-------------------|----------------------------------|-------------------------------------|------------------------------|
| Default           | build-time static                | per-request server                  | per-request<br>server → SPA  |
| Default<br>output | static HTML, 0 JS                | server-rendered HTML                | server-rendered<br>SPA shell |
| The<br>exception  | server island /<br>client island | static generation /<br>"use client" | static server<br>function    |

The exception is what you reach for to **break out of the default mode**.

# What has to justify itself?

|                       | What needs no justification    | What needs a directive                               |
|-----------------------|--------------------------------|------------------------------------------------------|
| <b>Astro</b>          | static HTML                    | anything server-per-request,<br>anything interactive |
| <b>Next.js</b>        | server rendering every request | static pages, client-side interactivity              |
| <b>TanStack Start</b> | SSR + SPA hydration            | frozen / prerendered pages                           |

# Default execution unit

At what granularity does the default mode apply?

|                       | Default execution unit                                    |
|-----------------------|-----------------------------------------------------------|
| <b>Astro</b>          | per component (build time)                                |
| <b>Next.js</b>        | per component (request time)                              |
| <b>TanStack Start</b> | per <b>route</b> (request time, then hydrates whole tree) |

- Astro and Next both isolate at the **component** — islands sit next to each other with different timing.
- Start treats the **whole route** as one server execution, then rehydrates all of it into the SPA.

# Static opt-in unit

At what granularity do you opt **out** of the per-request default?

|                       | Static opt-in unit                                                                                            |
|-----------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Astro</b>          | per component ( <code>server:defer</code> / <code>client:*</code> to <b>add</b> dynamism to a static default) |
| <b>Next.js</b>        | file / route ( <code>"use client"</code> , <code>force-static</code> , <code>generateStaticParams</code> )    |
| <b>TanStack Start</b> | <b>individual server function</b> ( <code>staticFunctionMiddleware</code> )                                   |

# Start's granularity is finer than its own route

```
// src/routes/index.tsx – one route, mixed timing
loader: async () => {
  const nav = await getNavTime();           // staticFunctionMiddleware → baked at build
  const greeting = await getGreeting();     // plain server fn → live per-request
  return { nav, greeting };
},
```

- `getNavTime` is frozen; `getGreeting` stays live — **on the same route**.
- In Next, the equivalent switch ( `force-static` ) is a file-level export that locks the whole route.
- Start's static unit is **finer** than the route default that contains it.

# The ordering that matters

| Astro            | Next.js                           | TanStack Start                  |
|------------------|-----------------------------------|---------------------------------|
| build-time first | server-first, but provably static | server-first, then client-owned |

**Astro:** nothing runs unless you say so.

**Next.js:** the server runs it, but tries to prove it didn't have to.

**Start:** the server runs it, then gives everything to the client.

## Side by side — the same four ideas

| Concept              | Astro                                | Next.js                       | TanStack Start                                     |
|----------------------|--------------------------------------|-------------------------------|----------------------------------------------------|
| Route                | <code>src/pages/index.astro</code>   | <code>src/app/page.tsx</code> | <code>src/routes/index.tsx</code>                  |
| Server-only render   | <code>.astro</code> component        | Server Component              | server function<br>( <code>createServerFn</code> ) |
| Interactivity        | <code>&lt;C client:load /&gt;</code> | <code>"use client"</code>     | every component                                    |
| Client → server call | <code>actions.foo()</code>           | <code>"use server"</code> fn  | <code>useServerFn(foo)</code>                      |



# Side by side — rendering defaults

|                       | Astro                            | Next.js                             | TanStack Start                                 |
|-----------------------|----------------------------------|-------------------------------------|------------------------------------------------|
| Default output        | <b>static</b>                    | static, dynamic where needed        | <b>SSR per request</b>                         |
| Opt out               | <code>prerender = false</code>   | dynamic API usage                   | <code>prerender: { enabled: true }</code>      |
| Deferred part         | <code>server:defer</code> island | <code>&lt;Suspense&gt;</code> + PPR | unawaited promise + <code>&lt;Await&gt;</code> |
| JS shipped by default | <b>none</b>                      | client components only              | <b>the whole app</b>                           |

## Side by side — the client ↔ server call

|                  | Astro                            | Next.js                   | TanStack Start              |
|------------------|----------------------------------|---------------------------|-----------------------------|
| Declared by      | <code>defineAction</code>        | <code>"use server"</code> | <code>createServerFn</code> |
| Input validation | <code>input: z.object()</code> ✓ | none ✗                    | <code>.validator()</code> ✓ |
| Errors           | <code>{ data, error }</code>     | <code>throw</code>        | <code>throw</code>          |
| HTTP method      | POST                             | POST                      | you choose                  |

# A fifth axis — cacheability

The three frameworks ship **very different things** on the wire. That determines what a CDN can cache, without any app changes.

- **Astro** — two independent HTTP requests: shell (static) + island (dynamic).
- **Next.js PPR** — one chunked HTTP response: static shell + streamed holes in the same connection.
- **TanStack Start** — one per-request response: no static/dynamic split in the transport at all.

# Server-side caching

|                           | What's cacheable                                        | Cached by                            | Mechanism                                                    |
|---------------------------|---------------------------------------------------------|--------------------------------------|--------------------------------------------------------------|
| <b>Astro</b>              | shell + island,<br>independently                        | any CDN incl.<br>nginx               | two separate HTTP<br>GETs                                    |
| <b>Next.js</b>            | shell (in theory)                                       | streaming-<br>aware edge<br>(Vercel) | one chunked<br>response, shell +<br>streamed holes           |
| <b>TanStack<br/>Start</b> | only<br><code>staticFunctionMiddleware</code><br>output | build output /<br>CDN                | no shell; page is<br>either fully live or<br>frozen at build |

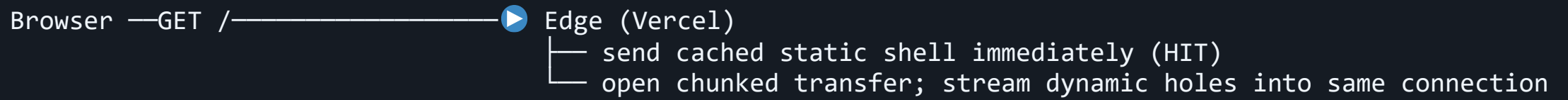
# Astro — two requests, two caches

```
Browser —GET /————▶ CDN (nginx, any CDN)
                        └─ HIT: serve cached index.html ← shell
                        └─ MISS: fetch from origin once, cache 10 min

Browser —GET /_server-islands/Greeting▶ Astro server (bypasses CDN cache)
                        └─ reads user-agent, renders Greeting
                        └─ returns HTML fragment
```

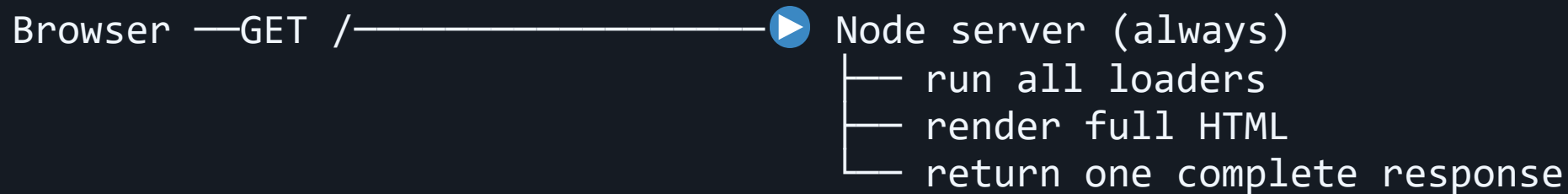
- The shell ( `index.html` ) and the island ( `/_server-islands/Greeting` ) are **different URLs** with different caching rules.
- nginx (or any reverse proxy) caches the shell the same way it caches any static file — no streaming support needed.
- The island can also be cached independently (e.g. 10s TTL) if it is not truly per-user.

# Next.js PPR — one response, shared pipe



- The capability is real — the framework supports it.
- But shell + dynamic content share **one HTTP response**. nginx has no concept of "cache part of this response and pass through the rest of the same connection."
- **This is not a missing config option** — it is a genuine feature gap between generic reverse proxies and PPR-aware edge infrastructure built into Vercel.

# TanStack Start — no middle tier



- There is **no shell/dynamic split** in the response — one document, generated fresh, every time.
- The only escape: pull data out of the per-request path via `staticFunctionMiddleware` — but that freezes at the individual function level.
- **Two modes, nothing in between:** fully live per-request, or function result frozen at build.

# A sixth axis — client navigation caching

This is separate from server caching. It governs what happens **after** the page is in the browser.

|                       | Client-side navigation caching                                                                                                          |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Astro</b>          | None — MPA by default; each nav is a fresh full page load                                                                               |
| <b>Next.js</b>        | Router Cache for prefetched segments (implicit, version-dependent)                                                                      |
| <b>TanStack Start</b> | Explicit SWR cache in the router: <code>staleTime</code> , <code>gcTime</code> , <code>preloadStaleTime</code> — configurable per route |

Start is the only one of the three that also gives you a fully explicit, tunable cache for what happens **after** the page arrives.



# Start's router cache — three timers

```
export const Route = createFileRoute("/")({
  loader: () => getGreeting(),
  staleTime: 60_000, // navigating back within 60s reuses cached data
  preloadStaleTime: 10_000, // hover re-preloads if older than 10s
});
```

- `preloadStaleTime` — governs **hover** (prefetch trigger). Default: 30s.
- `staleTime` — governs **click** (navigation trigger). Default: 0 (always refetch).
- `gcTime` — how long cached data is kept in memory after the route is left.

`staleTime: 0` is why every navigation reruns the loader even when the page was prerendered.

# Why Start's client cache exists

- Astro and Next treat "the app is a persistent client-side router" as the exception.
  - Astro: full page reload per navigation by default.
  - Next: implicit router cache, behaviour changes between versions.
- Start treats the client router as the **primitive** — SSR is the server delivering the first render *to* the SPA.
- So Start is the only framework here with a fully documented, per-route, explicitly configurable client cache — because it has to be: the SPA never stops running.

# A seventh axis — DX and mental models

All three frameworks require you to learn something new. The question is **what kind of new**.

- **Astro** — keeps React completely separate from its own templating layer.
- **Next.js** — makes ordinary-looking React files behave differently based on invisible directives.
- **TanStack Start** — replaces invisible magic with real, callable JavaScript functions.

**The single best DX lens:** is the framework's magic **implicit** (a convention you must memorize) or **explicit** (a function you can see, name, and trace)?

# Astro DX — two familiar things, kept separate

- The `.astro` component is closer to a classic template language (Handlebars, EJS, Pug) than to React.
  - `---` fence = data fetching. Template below = HTML output. No hooks, no lifecycle.
  - Fits your intuition immediately because it *is* the old-school model.
- The React island is **100% ordinary React** — because it is a fully separate render tree with its own bundle.
  - No new React concepts. `useState`, `useEffect`, callbacks — all work as expected.
- **The trade-off:** islands are separate trees. They don't share React state or context with the Astro shell or with each other.

# Next.js DX — familiar syntax, unfamiliar rules

- You cannot `import` a Server Component inside a `'use client'` file.
- The escape hatch is **composition-by-children** — not a higher-order component pattern:

```
// page.tsx (Server Component)
<ClientWrapper>
  <ServerChild /> { /* rendered on server, passed as a resolved slot */ }
</ClientWrapper>
```

- `ClientWrapper` never imports `ServerChild`; it just renders `{children}`.
  - The server resolved `ServerChild` before handing anything to the client tree.
  - Real pattern, but nothing in plain React trains you to think of `children` as a boundary-crossing mechanism.

# Next.js DX — the root cause

Why do both " `async` component / no hooks" and "can't import Server Component" feel foreign at the same time?

- Next repurposes **ordinary-looking React syntax** to mean something environment-dependent.
- A string literal at the top of the file ( `'use client'` / `'use server'` ) is intercepted by the compiler and silently changes what the file is allowed to do and where it runs.
- It looks identical to the React you know. It behaves differently.

**Astro's unfamiliarity** is a new file type with new syntax → easy to see.

**Next's unfamiliarity** is the same file type, same syntax, different semantics

→ harder to spot.

# TanStack Start DX — explicit over implicit

- `createServerFn(...).handler(...)` is a **real function** you call, wrap, compose, and pass around.
- No magic string. No compiler rewriting the meaning of a component file.
- The server/client boundary is drawn by **which function you're inside**, not by a hidden annotation on the file.

```
export const getGreeting = createServerFn({ method: "GET" })  
  .handler(async () => { ... });      // server boundary is here, in the call chain
```

- For someone from an SPA background: this is just "a hook that calls an API, with the network boundary handled for you."
- TanStack Router and Query are libraries SPA developers already have muscle memory for — Start reuses them rather than replacing them.

# Implicit vs. explicit magic

|                       | What you must learn                                                                                                        | Magic type                                                            |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| <b>Astro</b>          | <code>.astro</code> syntax, <code>client:*</code> directives, nanostores for cross-island state                            | Explicit — different file extension, tag-level directives you can see |
| <b>Next.js</b>        | Server Component rules, <code>children</code> as boundary, <code>'use client'</code> / <code>'use server'</code> semantics | Implicit — same React syntax, compiler changes meaning invisibly      |
| <b>TanStack Start</b> | <code>createServerFn</code> builder chain, loader/staleTime model                                                          | Explicit — real function calls you can read, name, and trace          |



# Which one?

| Use case                                           | Pick           |
|----------------------------------------------------|----------------|
| Content, docs, marketing, blogs                    | Astro          |
| Mostly-static pages with a few interactive corners | Astro          |
| Server-heavy app, big team, Vercel-shaped deploy   | Next.js        |
| Dashboards, editors, anything behind a login       | TanStack Start |
| You already love TanStack Query/Router             | TanStack Start |

The Todo app is ~130 lines in **all three**. The difference is not the code — it's **what ships**.